ICT3113 Assignment 2 – Performance Optimisation: Fast Triage Within an Accuracy Floor

This assignment is worth 20% of your final mark. Final submissions are due 2359 on 27th November.

Background

In Assignment 1 you built a ticket triage service, defined its performance and accuracy requirements, and evaluated three to five open-source models running on CPU-only Ollama. Two observations were common across the submitted reports, although you should confirm them against your own results rather than take them as given.

1. Model inference accounted for almost all of the request latency. The service itself typically added a few milliseconds per ticket, while the Ollama call added anything from two seconds to well over a minute depending on the model. Where this holds, further work on the service code has little effect on end-to-end latency.
2. The smaller models were quick but did not classify reliably, and the larger models were more accurate but too slow to sustain the required arrival rate on the available hardware.

Most teams concluded that no single candidate met all of the requirements they had set. The client has reviewed these findings and revised the requirements for the next phase.

Speed is now the client's primary concern. They are prepared to accept a lower accuracy than the best Assignment 1 model achieved, within a defined limit: the deployed system must remain within 10 percentage points of the top Assignment 1 model's accuracy on the team's golden set, and no individual category may fall too far. The limits are defined precisely in Step 1.

The constraints from Assignment 1 are unchanged. Inference must run on CPU, on hardware the team controls, with no hosted model APIs. The client has also made one thing clear: they will not buy more compute or memory for this. Whatever you recommend has to run on the same machine, with the same CPU and RAM, that you tested in Assignment 1.

The Task

Identify the fastest triage configuration that satisfies the client's revised accuracy limits, and support the recommendation with measurements taken using the Assignment 1 test method.

Every team tests these configurations:

- **Top model alone (required).** Your most accurate Assignment 1 model, unchanged. This is the reference: its accuracy sets the limit, and its latency is what you are trying to beat.
- **Each other Assignment 1 model alone (required).** Every other Assignment 1 candidate, unchanged. A smaller model may already pass the accuracy limits and be the fastest option. Check it; do not assume.
- **One cascade, at two settings (your design).** A smaller model classifies first. A rule decides whether to accept its answer or pass the ticket to a larger model. This is the new configuration. It may or may not beat the best single model.
- **The cascade has one hard constraint.** Both models must be loaded in memory at the same time, on the same machine you used in Assignment 1, with no extra RAM or CPU. In Assignment 1 you only ever had one model loaded. Some teams' machines will not hold their top model and a smaller model together. Check this first (see Step 2, Memory) before you design anything, because it decides which pairs you can even try.

A cascade incurs the cost of the small model on every ticket and the cost of the large model on the tickets it escalates. Ignoring queueing and routing overhead, its mean latency is approximately the small-model latency plus the escalation fraction multiplied by the large-model latency. Whether this is lower than the large model alone therefore depends on both the speed of the small model and how often escalation occurs. The escalation fraction must be reported from the service logs for every cascade result, alongside the latency figures. As an illustration with made-up numbers: if the small model takes 2 s and the large model takes 6 s, a cascade that escalates 30% of tickets averages about $2 + 0.3 \times 6 = 3.8$ s, which is faster than the large model alone; the same cascade escalating 70% of tickets averages about $2 + 0.7 \times 6 = 6.2$ s, which is slower.

It is acceptable for a single mid-sized model to be the recommended configuration, and it is acceptable to conclude that the cascade did not improve on it.

Scope: What Is Fixed and What May Change

The following are fixed from Assignment 1:

- The candidate models, identified by the same Ollama tags and digests. No new models may be introduced.
- The classification prompt and the seven categories. The same prompt is used for every model in every configuration.
- The golden test set and its labels, which were frozen in Assignment 1.
- The service endpoints (POST /tickets, GET /search, GET /stats) and their behaviour.
- CPU-only Ollama on hardware the team controls.
- The test method: open-loop load generation with Apache JMeter, three runs per configuration, with raw .jtl files and service logs retained in the repository.
- The test machine, including its CPU count and the RAM given to Docker and to the Ollama container. The client is not adding compute or memory, so neither do you. If you must change machines, report it and explain why.

The following may change:

- The implementation of POST /tickets, so that it can call one model, apply a rule, and call a second model where required.
- The escalation rule and whatever it depends on, for example a second prompt requesting a confidence indicator, a second small model for comparison, or a check on ticket length. The rule may introduce an additional model call, but the classification prompt itself is unchanged.
- The pairing of models within the cascade, provided both are Assignment 1 candidates.

The following are not permitted:

- Caching of classification results for previously seen tickets, since the golden set would then be answered from the cache.
- Changes to Ollama's inference settings (thread count, parallelism, context length) between configurations. These should be recorded once and held constant.
- Fine-tuning, additional classifiers, retrieval, or any other component that was not part of the Assignment 1 system.

Teams that are unsure whether a proposed change is within scope should ask before collecting measurements.

Step 1: Define the Accuracy Limits

Split the golden set. Sort your golden-set tickets by row number. Odd rows are your tuning set. Even rows are your test set. Commit this split before you do anything else.

Use the tuning set to design and adjust your escalation rule. Use the test set for every accuracy number you report. Do not tune on the test set. If you pick a rule using the same tickets you then score it on, the score tells you nothing.

One honest caveat: all your models already ran on the full golden set in Assignment 1, so the test set is not new data. What it gives you is a fair comparison of all configurations on the same tickets, with the cascade rule not tuned on them.

Overall limit. Run the top Assignment 1 model on the test set. The overall accuracy limit is its test-set accuracy minus 10 percentage points, not the full-golden-set figure from your Assignment 1 report. For example, a top model scoring 74.4% on the test set gives a limit of 64.4%. In practical terms, this means the client has agreed to accept a system that classifies 64.4% of tickets correctly, provided it is faster.

Per-category limit. For each of the seven categories, no configuration may score more than 15 percentage points below the top model's accuracy on that category, measured on the test set. This prevents a configuration from satisfying the overall limit by abandoning one category. The Assignment 1 requirement of 70% per category does not apply in Assignment 2.

Small numbers. The test set has about 90 tickets, so small differences between configurations do not mean much. Some categories will have fewer than 10 tickets; in a category with 8 tickets, one ticket is worth 12.5 points. Show counts (right/wrong) next to every percentage, and say when a category is too small for the per-category limit to be a fair test.

Step 2: Design the Cascade

In a cascade, every ticket is first classified by a smaller model. A rule then determines whether that classification is accepted or whether the ticket is passed to a larger model, whose classification is final.

Small generative models do not provide a dependable confidence score, so the signal used for the decision has to be constructed. Teams should select one approach, justify the choice, and hold it fixed during measurement. Approaches that could be considered include the following.

Example method 1: Category-based. Accept the small model's answer when its predicted category is one it gets right reliably; otherwise escalate. Decide which categories count as reliable from the small model's results on the tuning set only, not from your Assignment 1 full-golden-set results, because those included the test tickets. The true category is unknown when the ticket arrives, so the rule can only use the predicted one. Check how often a wrong answer slips through because the small model predicted a "safe" category.

Example method 2: Agreement. Two small models (or one model asked twice with different wording) classify the ticket. Escalate if they disagree. Two small calls may cost as much as one large call; measure it.

Example method 3: Self-reported confidence. Ask the small model for a category and HIGH or LOW confidence. Escalate on LOW. Easy to build, but small models are often confident and wrong. If the confidence comes from a separate model call, that call is part of the cascade and its time counts.

Example method 4: Ticket length. Short tickets go to the small model, long tickets straight to the large model. Fast to decide. Only useful if length actually predicts errors in your data; check against your Assignment 1 results.

Teams may design a different rule. In every case the rule must be documented clearly enough that another team could implement it from the description.

Most rules have a setting you can turn: a list of categories, a length cut-off, a confidence word. Different settings give different escalation fractions, and so different accuracy and speed. Choose your two settings using the tuning set, then report both on the test set so the trade-off is visible. Two settings, not more; the experiment has to fit in the time you have.

The cascade is implemented within POST /tickets. The endpoint continues to accept one ticket and return one category, and the caller should not be able to tell which model produced the answer. The service log must record, for each ticket, which model produced the final classification and whether escalation occurred, so that the escalation fraction can be derived from the logs.

Memory: both models must fit at the same time. A cascade only works if the small model and the large model are both loaded in memory together. If they do not both fit in the RAM available to Ollama, Ollama will unload one to load the other every time a ticket is escalated, and each escalation pays several seconds of model loading on top of inference. That is not a cascade result; it is a loading result. So the rule is: a cascade is only tested with both models resident. Check this before you build anything: load both models, run a few tickets through each, and confirm that `ollama ps` lists both. If they do not both stay

loaded, try a smaller pair. If no pair fits, report that a cascade is not feasible on this machine, show the `ollama ps` evidence, and recommend a single model. Asking for more RAM is not an option; the client has said no.

Step 3: Record Predictions

Before any testing, record and commit the team's predictions. As in Assignment 1, the commit history serves as evidence that predictions preceded measurement. Predictions are not revised afterwards.

Predictions should be specific enough to be shown wrong, and should cover:

- which configuration is expected to be the fastest that satisfies the accuracy limits, with reasons;
- for the cascade, the expected escalation fraction, accuracy and median latency at each setting to be tested;
- which categories are expected to lose the most accuracy when the small model's answer is accepted.

Marks are awarded for the specificity of the predictions and for the later analysis of where they were wrong, not for their correctness.

Step 4: Test and Measure

Two kinds of test are run on each configuration, using the Assignment 1 method.

Accuracy test. Run every test-set ticket through POST /tickets, once per configuration, and score the answers against your golden labels. Report three things: overall accuracy (count and percentage), accuracy per category (checked against the per-category limit), and a confusion matrix.

For the cascade, also report how many tickets were escalated, and the accuracy of the escalated and non-escalated tickets separately. This shows whether your rule is catching the tickets the small model gets wrong.

You may also show the tuning-set results and compare them with the test-set results. Only the test-set results are used to decide whether a configuration passes.

For single models you may reuse your Assignment 1 prediction records instead of re-running, as long as nothing has changed (same digest, prompt, Ollama version and machine). Score only the test-set rows, and say in the report that you did this. Speed

results from Assignment 1 cannot be reused; see below.

Reusing single-model accuracy does not let you skip the cascade. Even if you end up recommending a single model, you must still build the cascade, test its accuracy, and run it through the speed test, so that you can show the single model is in fact faster. If the cascade fails the accuracy limits, still report its speed, marked as not passing accuracy.

Speed test. Do the accuracy test first. Then speed-test every configuration that passes both accuracy limits, and the cascade in all cases. Single models that fail accuracy do not need a speed test. Everything that is speed-tested, including the top model alone, is measured under the same Assignment 2 conditions:

Set-up

- Set the arrival rate from your top model's speed, not from the Assignment 1 client rate. Teams' Assignment 1 rates ranged from 19 tickets an hour to several a minute; at the low end a run would contain only a handful of requests, too few to measure anything.
- How to set it. Restart Ollama and do the normal warm-up. Then send three different tickets of typical length to your top model, one at a time, and take the middle time. Do not send the same ticket three times: Ollama caches prompts it has seen, and a repeat comes back much faster than a new ticket would. Call the middle time S, in seconds. Say S is 6. Double it: 12. Send one ticket every 12 seconds, which is 5 per minute. This leaves the top model idle about half the time, so no queue builds.
- How long to run. Run for S minutes, with a minimum of 5. With S = 6 that is 6 minutes, about 30 tickets. With S = 34 it is 34 minutes, also about 30 tickets. Every team ends up with roughly 30 tickets per run, whatever their machine.
- Use the same rate and run length for every configuration, including the single models. Write S, the rate and the run length in your report, with the three timings and the three ticket row numbers you took S from.
- Compare your rate with the client's peak rate from your Assignment 1 workload model. If your test rate is above the client's peak, your result also shows the system can handle the client's load. If it is below, your result compares the configurations but does not show that the recommended one can handle the client's peak. Say which case you are in, on the limitations slide.
- Why the queue matters: if tickets arrive faster than the model can finish them, the queue gets longer for the whole run, and so does the waiting time. The p50 you measure then depends on how long you ran, not on the configuration. A 5-minute run gives one number and a 10-minute run gives a bigger one. You cannot compare

configurations on numbers like that. At a rate the model can keep up with, the latency settles and stays put, so the number describes the configuration and the comparison is fair.

- Plan three runs per configuration. One run is not enough; the spread across runs is part of the result.
- Use one fixed set of tickets, in the same order, for every configuration. Take them from your rows outside the golden set. Include some typical and some long tickets. Commit the list of row numbers.
- Before the first run, record your software versions, model digests and Ollama settings. Check they are unchanged before each configuration.

Each run

- Restart the Ollama container and check it is up (GET /health on your service, or `ollama ps` in the container). This clears Ollama's prompt cache. No model is loaded yet.
- Send three fixed warm-up tickets, one at a time. The first one loads the model. Throw away all three timings. Use the same three tickets every time.
- For the cascade, warm up both models. Make sure the warm-up tickets reach the large model too (for example, pick warm-up tickets your rule will escalate, or send one directly to the large model). Then run `ollama ps` in the container and confirm both models are listed. Do not start the run until they are.
- Call GET /stats on your own triage service (the Assignment 1 endpoint that returns the total ticket count) and note the total.
- Run the JMeter test at your rate and run length from Set-up.
- Let slow requests finish. Two settings matter, and the order matters. First, the HTTP sampler's response timeout: set it to 10×S seconds, and at least 5 minutes. Second, the pause at the end of the Open Model Thread Group schedule: set it longer than the timeout by at least one minute, for example `rate(5/min) random_arrivals(6 min) pause(6 min)` for S = 6 with a 5-minute timeout. The pause keeps JMeter alive without sending anything. When the schedule ends JMeter kills any request still running, so the pause must outlast the timeout; otherwise a slow request can be cut off and lost, and those are the slow ones you most need. Even with this, check the counts below for unfinished requests.
- Call GET /stats on your triage service again. The increase in the total is the number of requests the service completed and stored.

Report, per run

- You should have three runs for every configuration. Report all three.
- Requests sent, completed, errored and unfinished. A run with unfinished requests is reported as such, not thrown away.
- Latency comes from the JMeter .jtl file: each request has an elapsed time. Sort the elapsed times of the successful requests. p50 is the middle value; p95 is the value 95% of the way up.
- Report p50 for each run, with the mean and spread across the three runs.
- A single run of about 30 requests is too small for a stable p95 (it is roughly the second-slowest request). Compute p95 on the three runs pooled together, about 90 requests, and state the number it is based on. Do not report p99; the sample is too small.
- Achieved throughput (completed requests per second) and error rate, per run.
- For the cascade, the escalation fraction from the service logs. It may differ from the accuracy-test figure, because the tickets are different. Explain any large gap.

Do not reuse Assignment 1 speed figures for any configuration, including the top model. Everything is re-measured under these conditions.

Keep all raw .jtl files and service logs. Every figure in your slides must trace back to them.

Test environment. Describe the test machines as in Assignment 1. If the machines are unchanged, state this and refer to the Assignment 1 description. The load generator must run on a separate machine from the service.

Step 5: Recommendation

Recommend one configuration for deployment. It must satisfy both accuracy limits on the test set and must be the fastest of those that do, according to the team's own measurements at the chosen arrival rate.

State how much faster it is than the top model alone and how much accuracy it gives up overall and by category.

If the cascade was not the fastest qualifying configuration, the report should say so and explain why, with reference to the measured escalation fraction and latencies.

If no configuration other than the top model alone satisfied the limits, the recommendation is the top model, and the report describes what was attempted and what it cost.

Deliverables

Submission format

- One PowerPoint file of at most 8 slides, together with the supporting files listed below.
- Submit via xSiTe by 2359 on, Friday 27th November.
- File name: GroupNum_A2.pptx, for example Group01_A2.pptx.
- Supporting files: the prediction record (Step 3), the written escalation rule (Step 2), and the list of row numbers used as the speed-test workload.

The prediction record and the escalation rule must be committed to the repository before the first Assignment 2 benchmark run. Raw .jtl files and service logs for every reported run are retained in the repository and may be requested for inspection.

Slide requirements

Slide	Content
1. Cover	Group, names, IDs, repository link. Top model and its Assignment 1 accuracy. Overall and per-category limits.
2. Cascade design	Diagram. Escalation rule in full. Settings tested. Why this rule.
3. Test environment and method	Machines (or refer to Assignment 1). Arrival rate and why you chose it. Workload tickets. Warm-up and drain procedure. Any change from Assignment 1.
4. Accuracy results	Overall and per-category accuracy for every configuration, with counts. Which pass the limits. Escalation fraction; accuracy of escalated vs non-escalated tickets.
5. Speed results	p50 per run with spread, pooled p95, throughput and error rate for every speed-tested configuration. Escalation fraction under load.
6. Trade-off	One chart: accuracy vs latency, one point per configuration and setting, overall limit drawn as a line. Short interpretation.

Slide	Content
7. Predictions, recommendation, defence	Predictions vs outcomes and why they differed. Recommended configuration, speed gain, accuracy cost. If the cascade lost, why.
8. Limitations, references, AI declaration	What the results do not show (sample size, one machine, one arrival rate, seen tickets). References. Model licences. AI use.

Every figure on slides 4 to 7 must be traceable to a log or .jtl file in the repository.

Late submissions are penalised at 15% per day. Submissions more than four days late are not accepted.

Assessment Criteria

Submissions are assessed on the soundness of the cascade design, the completeness and reliability of the measurements, and whether the recommendation follows from the evidence. The 20% is allocated as follows.

Component	Criteria
Design (Step 2), 6%	Cascade inside the existing service. Rule documented so it can be reproduced. Two settings tested. Memory check done and reported. Logs show which model answered each ticket.
Measurement (Steps 1, 3, 4), 7%	Limits derived from the test set. Predictions committed before testing. Accuracy for every configuration. Three-run open-loop speed tests for every qualifying configuration and the cascade. Escalation fraction from logs. All figures reconcile with raw data.
Recommendation (Step 5), 7%	Fastest qualifying configuration identified from the team's own measurements. Trade-off shown. Predictions compared with outcomes. Clear statement if the cascade did not help. Limitations stated, including what the test set and test rate do and do not show.

Copyright, AI Use and Plagiarism

The notes from Assignment 1 apply. Acknowledge the Consumer Complaint Database, Ollama, and the licences of the models used.

AI coding tools may be used and are expected. They may be used to write the cascade code and the JMeter test plan. The escalation rule, the running of the tests, and the recommendation are the team's responsibility. The AI declaration should state what the tools produced and what the team did.

Submissions must be the team's own work. Fabricated or irreconcilable measurement figures are treated as an academic integrity matter rather than a marking deduction.

A Note on the Delay

This brief was issued later than planned, and I want to explain why.

The original intention for Assignment 2 was to have you optimise the service you built. Your Assignment 1 results showed that there was little to gain from this. In the reports I read, Ollama accounted for seconds of each request and the rest of the service for milliseconds, so the service code was not where the time was going.

The next idea was to vary the CPU allocated to Ollama. From what I saw in your reports, most teams were already using all the CPU their machines had, so the only available change was to reduce it, and the outcome would have been much the same for everyone: less CPU, slower responses.

The scope for optimisation turned out to be narrower than I expected when I wrote Assignment 1. The question that remained is the one set here: since model inference is where the time goes, can the models you already have be arranged so that the system is faster, without giving up more accuracy than the client will accept? I do not know the answer in advance, and I expect it will differ between teams.

Thank you for your patience, and for the work you put into Assignment 1.